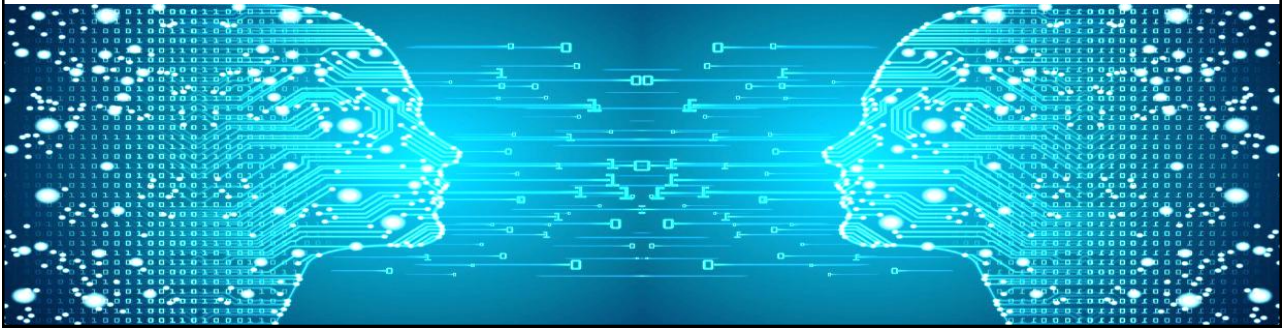
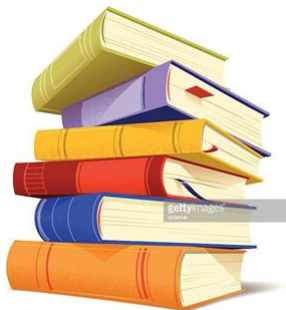


Machine Learning



Rule-Based Classifiers



- Introduction to Rule-Based Classifiers
- Sequential Covering Algorithm
- Visualization
- Ripper

Introduction to Rule-Based Classifiers

- Definition: A classifier that uses **if-then rules** for classification.
- Rules are human-readable and interpretable.
- Example:
 - Rule 1: *If outlook = sunny AND humidity = high THEN play = no*
 - Rule 2: *If outlook = overcast THEN play = yes*
- Key properties:
 - Easy to understand
 - Compact representation
 - Good for explanation

Overview

- A common method to build rule-based classifiers.
- Procedure:
 - Learn one rule that covers part of the data.
 - Remove instances covered by that rule.
 - Repeat until all instances are covered.
- Also called **Separate-and-Conquer**.

Pseudo-code

Input: A labeled Data set
 Output: A Set of IF-THEN rules.
 Method:

1. Start with an empty **Ruleset named RS**.
2. For each class **c**, perform the following steps:
 - Repeat
 - i. Use **Learn-One-Rule** to find the best rule **R**.
 - ii. Remove examples/tuples covered by **R**.
 - Until the terminating condition is satisfied. //e.g. no tuples left or quality of rule is below specific threshold.
 - Add **R** to **RS**.
3. Return: **RS**.

Visualization

- Rules can be **visualized** for interpretability.
- Example (Fig. 3.1, p.84): Sheep vs Goats classification.
 - Simple rules based on features (e.g., ear length, color).
- Visualization helps in understanding which conditions lead to classification.

Ripper (Introduction)

- **RIPPER:** Repeated incremental pruning to produce error reduction
- A widely used rule learner.
- Steps:
 1. Rule Growing
 2. Rule Pruning
 3. Optimization

Ripper Algorithm

- Training process:
 - Grow rules greedily using **information gain**.
 - Prune rules to avoid overfitting.
 - Repeat until data is covered.
- Produces compact, optimized rule sets.

Ripper Algorithm

- Pseudo-code (simplified):

For each class:

While positive examples remain:

Grow rule using conditions with max information gain

Prune rule if necessary

Add rule to rule set

Decision Tree vs Rule-Based Classifier vs RIPPER

Feature	Decision Tree	Rule-Based Classifier	RIPPER
Representation	Tree structure (nodes & branches)	Set of if-then rules	Optimized set of if-then rules
Interpretability	Easy to read, but large trees are hard to follow	Very clear (explicit rules)	Clear, concise, simplified
Construction Process	Split data on best attribute	Sequential covering algorithm	Rule growing → Pruning → Optimization
Overfitting Risk	High without pruning	Medium	Controlled by pruning & optimization
Accuracy	Often high, depends on pruning	May be lower than trees/ensembles	High, robust on real-world data
Applications	Suitable for small and large datasets	Best for explanation and reporting	Real-world tasks needing balance between accuracy & interpretability

Rule Growing Process

- Rules grown by adding conditions that maximize **information gain**.
- Example: Start with empty rule → add best attribute → refine further.
- Stop when rule covers mostly positive instances.
- **Python Example:** Section 3: Rule Growing

Information Gain

- Measure of “**purity improvement**” when adding a condition.
- Formula (Entropy-based):

$$Gain(S, A) = Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

- Used in rule-growing (similar to decision trees).

Pruning

- Purpose: avoid overfitting.
- Process: remove conditions that don't improve performance on validation set.
- Example: Rule "If ear length > 10 AND color = white" → Prune to "If color = white".

Optimization

- RIPPER performs optimization after pruning:
 - Reorders rules.
 - Replaces or revises rules to minimize error.
- Goal: balance **accuracy** and **rule simplicity**.

Summary

- Rule-based classifiers: interpretable models using **if-then rules**.
- Built using **Sequential Covering** or advanced algorithms like **RIPPER**.
- Key steps: Rule Growing → Pruning → Optimization.
- Advantages: interpretability, simplicity.
- Limitations: may not achieve best accuracy vs. complex models.

